

Association Transformations in Multi-tier Structures

1. Introduction

This paper presents a method for learning transformations over multi-tier representations, with a particular focus on the modification of cross-tier relations (i.e., associations between tiers), which is widely used in linguistics. We consider a specific but linguistically important type of transformations where the nodes remain unchanged from input to output, while only the association relation is modified subject to non-crossing constraints. We formalize these representations as temporally-ordered tiers with a binary association relation, and the learning problem is therefore framed as inferring the which association relations are preserved, deleted, or newly introduced from observed input-output pairs.

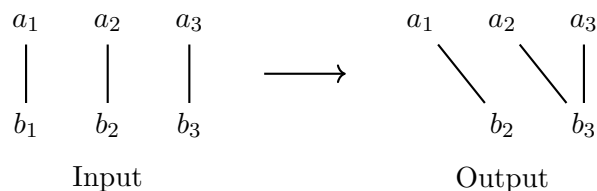


Figure 1: Learning association transformations between tiers.

Such multi-tiered structures have been widely used in linguistic theory, such as tone-syllable associations in autosegmental phonology (Goldsmith, 1976) prosodic or syllable structure (Hayes, 1980; Sagey, 1986). This insight implies that Linguistic information of different aspects are organized across multiple temporally ordered tiers. Associations between tiers encode which elements are linked at a given point in time, thereby capturing relationships across different dimensions of the structure. We refer this type of graph as *autosegmental graphs* (ASGs), which can be viewed as bipartite graphs, where nodes with shared properties are organized into the same tiers. Since each tier is linearly ordered, associations are subject to non-crossing constraint (NCC, Bird and Ellison, 1994; Goldsmith, 1976). The core idea behind autosegmental graphs is that input-output mappings are in fact more naturally characterized by rearranging association lines across tiers rather than changing elements themselves. A large body of theoretical work on African tonal processes has demonstrated the advantages of autosegmental graphs (?). Computationally, using multi-tiered structures has been shown to reduce the complexity of tonotactic patterns (?).

Previous work has modeled these structures using finite-state approaches (Wiebe, 1992; Bird and Ellison, 1994; Yli-Jyrä, 2013, 2015). These approaches typically convert structured representations into strings or synchronized tapes to leverage finite-state machinery, or rely on hard-coded mechanisms to represent different types of associations.

In contrast, we propose an approach that learns association transformations between tiers directly over autosegmental graphs using model theory and formal logic. As a preliminary study, we begin with an idealized assumption: tonal processes involve no changes to

segmental content, but only the rearrangement of association lines between the underlying representation (UR) and the surface representation (SR). In addition, we assume that only a single process constitutes the learning target.

We show that, by adopting autosegmental graphs within a model-theoretic framework of formal transductions, it is possible to construct a partial order over structures and apply a model-theoretic learner such as the Bottom-Up Factor Inference Algorithm (Chandlee et al., 2019; Rawski, 2021). The target output of the learner is a formula that preserves unchanged associations, deletes associations that must be delinked, and introduces newly added associations. Learning proceeds through iterative hypothesis updates while traversing a partially ordered space of logically possible structures.

Furthermore, transformations that appear non-deterministic when modeled over strings can instead be expressed deterministically within this relational framework. To our knowledge, the problem of learning local transformations of association relations in multi-tier structures has not previously been addressed in this form.

2. Preliminaries

2.1. Autosegmental graphs

Definition 1 (Autosegmental Graph) *An autosegmental graph (ASG) is a labeled bipartite graph with multiple linearly-ordered tiers. Equivalently, it can be defined model-theoretically as a relational structure $\mathcal{A} = \langle D, \mathcal{T}, \alpha, \triangleleft \rangle$ where D is a finite domain of elements; each tier $T_i \subseteq D$ is a set of nodes. $\mathcal{T} = \{T_1, \dots, T_n\}$ is a collection of tiers such that $T_1 \cup T_2 \cup \dots \cup T_n = D$ and $T_i \cap T_j = \emptyset$ for all $i \neq j$. $\triangleleft$ is a binary successor relation defined between nodes on the same tier; $\alpha \subseteq D \times D$ is a set of non-crossing association relations across tiers.*

ASGs represent the idea that different aspects of linguistic structure are organized on distinct tiers while progressing simultaneously over time. Each tier encodes a particular type of information, such as tone (pitch information), segmental (tongue shape), syllabic (metrical) structure. Association relations link elements across tiers that are realized at the same temporal position.

Note that there may be multiple tiers in ASGs, provided that the elements on different tiers are disjoint. In most phonological applications, ASGs typically involve two or three tiers. In this work, we assume a two-tiered structure, though the definition remains the same in the general case. In addition, ASGs are subject to the No Crossing Constraint (NCC), which enforces temporal alignment between associations.

Definition 2 (No Crossing Constraint (NCC)) *An ASG needs to obey No Crossing Constraint. Formally, For all $(x, y), (x', y') \in \alpha$, $x \triangleleft x' \Rightarrow (y \triangleleft y') \vee (y = y')$. and vice versa.*

In addition to representing model-theoretic relational structures with graphs, one can use *adjacency matrices* which use binary values to denote whether there is an edge between two nodes. Two nodes are considered *adjacent* if there is an edge connecting them.

Definition 3 (Adjacency matrix) *Given an ASG $\mathcal{A} = \langle D, \mathcal{T}, \alpha, \triangleleft \rangle$ and two tiers $T_i, T_j \in \mathcal{T}$, the association relation α can be represented as a binary matrix where $M_\alpha[k, \ell] = 1$ if $(x_k, y_\ell) \in \alpha$ where $x_k \in T_i, y_\ell \in T_j$; otherwise 0.*

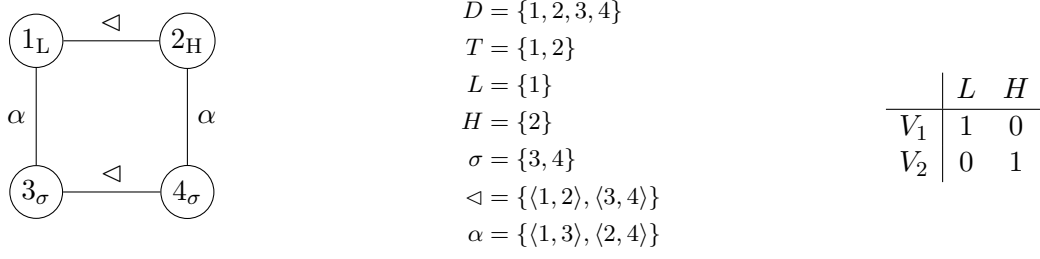
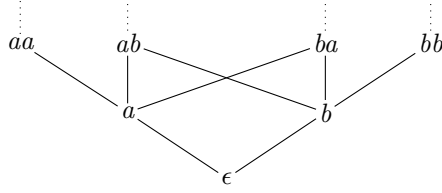


Figure 2: Three representations of a two-tier autosegmental graph.


 Figure 3: Strings over $\Sigma = \{a, b\}$ ordered by the containment relation (non-exhaustive)

The motivation for adopting matrices is to establish a framework corresponding to string-based learning. Concepts such as substrings, subfactors, and superfactors are well developed and widely used in grammar inference [NEED REF]. To incorporate ASGs into this framework, we follow the notions of *restriction* and *containment* proposed in Chandlee et al. (2019), adapting them to graph-based representations through the notions of submatrices and matrix containment.

Definition 4 (Submatrix; comparable restriction in Chandlee et al. (2019)) Let matrix A be an $m \times n$ matrix. A matrix B is a submatrix of A if it is formed by taking k ($1 \leq k \leq m$) contiguous rows and v ($1 \leq v \leq n$) contiguous columns from A .

Definition 5 (Containment) An $m \times n$ matrix $A = [a_{ij}]$ is contained in a matrix B (written $A \sqsubseteq B$) if there exists an $m \times n$ submatrix of B , denoted $B' = [b'_{ij}]$, such that for all $i \in [0, m]$ and $j \in [0, n]$, $b'_{ij} \geq a_{ij}$.

The containment relation provides a partial ordering over the space of all model-theoretic structures. As illustrated by the inverted pyramid, more general factors appear lower in the ordering, while more specific factors appear higher positions. This structure provides the foundation for the learning algorithm, which determines whether a structure is contained in another. Importantly, all factors that contain a forbidden structure, i.e., its superfactors, are also forbidden, while all factors contained in a well-formed structure, i.e., its subfactors, are also well formed.

Definition 6 (Neighbor) Given an ASG $\mathcal{A} = \langle D, T, V, \alpha, \triangleleft \rangle$, a node n is a neighbor of a node m iff $m \triangleleft n \vee n \triangleleft m$.

2.2. Logical Transduction

Using model theory (Strother-Garcia, 2019; Yolyan, 2025; Jardine, 2017), we model a phonological transformation from an underlying representation (UR) to a surface representation (SR) as a relation between input and output structures $\mathcal{A}_i \rightarrow \mathcal{A}_o$. There have been work adopting the same fashion on string-to-string transduction [NEED REF], but the essentially the logic is not restricted to strings and can operate over any model-theoretic representations. In this framework, we use a set of logic formula to derive the output signature with reference to the input signature. Recall from the previous section that we defined an ASG signature as $\mathcal{A} = \langle D, \mathcal{T}, \alpha, \triangleleft \rangle$. Given an input model with this signature, the output signature can be defined in terms of the input structure.

Output	Input	Interpretation
ϕ_{Domain}	$:= \text{TRUE}$	Preserve all input elements
COPY	$:= 1$	Define Output domain the same size as Input
$\phi_{\text{license}}(x)$	$:= \exists y \alpha(x, y)$	Preserve only associated elements
$T'(x)$	$:= T(x)$	Preserve element labels
$\triangleleft'(x, y)$	$\approx \triangleleft(x, y)$	Preserve linear order within-tier
$\alpha'(x, y)$	$:= (\alpha(x, y) \wedge \neg \text{deassoc}(x, y)) \vee \text{reassoc}(x, y)$	Update association relations

Table 1

As shown in Table 1, the transduction defines an output model with the same domain as the input structure, while filtering out unassociated elements. All unary predicates $T(x)$ in the input signature are preserved in the corresponding output predicates $T'(x)$, meaning that an element x has property T in the output iff it has property T in the input. Likewise, the within-tier order relation is preserved. The only change is the association relation α , which is defined such that an association line holds in the output iff either (i) it is present in the input and not removed by **deassoc**, or (ii) it is introduced by **reassoc**.

$$\begin{aligned} \alpha'(x, y) &:= (\alpha(x, y) \quad \wedge \quad \neg \text{deassoc}(x, y)) \quad \vee \quad \text{reassoc}(x, y) \\ \text{output} &= \text{unchanged} \quad - \quad \text{delinked} \quad \quad + \quad \text{newly added} \end{aligned}$$

In this way, the problem of learning association-line transformations can be reduced to learning the logical formulas for **deassoc** and **reassoc**. Since these two predicates are combined through disjunction, two parallel learning processes take place simultaneously, and their outputs are combined to define the final association relation α' .

We demonstrate this process using a common phenomenon in tone languages known as *contour formation*. When two vowels become adjacent, the first vowel (V_1) may be deleted. Since the tone originally associated with V_1 cannot remain unattached, the deletion triggers delinking of T_1 from V_1 and reassociation of T_1 to the following vowel V_2 . As a result, the surviving vowel bears both tones in their original order, which together surface as a contour tone.

From a logical perspective, this transformation can be characterized through the interaction of the **deassoc**, **reassoc**, and ϕ_{license} functions. The **deassoc** operation removes the

association between the deleted vowel and its tone, indicated by the crossed-out line in Figure 4. The **reassoc** operation then links the tone of the deleted vowel (T_1) to the surviving vowel (V_2), indicated by the dotted line in Figure 4. Consequently, T_1 and V_1 are no longer associated and therefore are not licensed by ϕ_{license} (shown in gray in Figure 4). Thus, T_1, T_2 and V_2 remain in the surface representation. If we define the relevant context as $\text{hiatus}(x, y) := V(x) \wedge V(y) \wedge \triangleleft(x, y)$, where x is the first vowel (V_1) and y is the second vowel (V_2), then the transformation can be expressed as follows:

$$\begin{aligned} \text{deassoc} &= \alpha(t, v) \wedge \text{delete_V}(v) \\ \text{reassoc} &= \text{survive_V} \wedge \exists v (\alpha(t, v) \wedge \text{delete_V}(v)) \end{aligned}$$

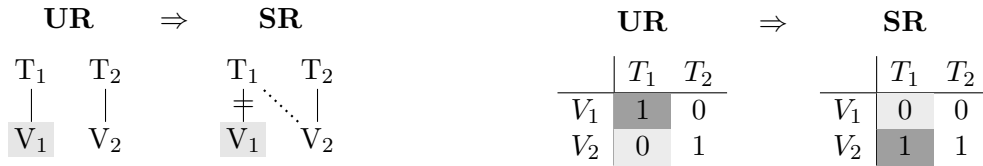


Figure 4

The **deassoc** and **reassoc** operations are equivalently reflected in the adjacency matrices. Figure [NEED REF] shows the input and output matrices corresponding to the structures in Figure 4. As can be seen, the value corresponding to the association between T_1 and V_1 changes from 1 in the input to 0 in the output, reflecting delinking. Conversely, the value corresponding to T_1 and V_2 changes from 0 to 1, reflecting reassociation. In this way, the autosegmental graph, the adjacency matrix, and the logical expressions provide equivalent description of the same transformation. Formally, **deassoc** and **reassoc** can also be identified by $\text{deassoc} = \{(i, j) \mid \text{UR}_{ij} = 1 \wedge \text{SR}_{ij} = 0\}$ and $\text{reassoc} = \{(i, j) \mid \text{UR}_{ij} = 0 \wedge \text{SR}_{ij} = 1\}$.

2.3. Association Learning

This section follows the framework of using logical transductions to learn phonological mappings in Yolyan (2025), which leverages the entailment relation in a partially ordered space and updates hypotheses about the smallest contextual environments that trigger a featural change (a string-based input-output mapping that can be written as $A \rightarrow B / C_D$).

Several aspects of this framework are particularly important for the present work. First, phonological processes are represented as logical formulas over model-theoretic representations. Second, the learning procedure relies on grammatical entailment relations within a partially ordered hypothesis space, with more general structures at the bottom and more specific structures higher up (Chandlee et al., 2019; Rawski, 2021). If an environment (which itself is a structure, thereafter we call it environment) is sufficient to trigger a transformation, then any superfactor containing it also entails that transformation. On the other hand, if an environment fails to trigger the change, then all of its subfactors are likewise insufficient to trigger the change. In other words, given a hypothesized environment, all data satisfying this condition are expected to undergo the transformation (positive evidence), while all data not satisfying this condition are expected not to undergo the transformation (negative evidence). If some positive data fail to undergo the transformation, then

the hypothesis is too restrictive. Conversely, if some negative data nevertheless undergo the transformation, then the hypothesis is too general. The goal of learning is therefore to identify the smallest common environment that are contained in all positive evidence, but absent in all negative evidence. For example, post-nasal voicing can be written as $[-\text{voi}] \rightarrow [+ \text{voi}] / [+ \text{nasal}, -\text{cont}] _$, meaning that whenever a segment specified as $[-\text{voi}]$ occurs after a $[+ \text{nasal}, -\text{cont}]$ segment, its voicing feature changes to $[+ \text{voi}]$.

In autosegmental phonology, however, such contextual environments are not expressed linearly as $A \rightarrow B / C _ D$, but instead as structural relations over graphs. Nevertheless, because the learning framework applies generally to model-theoretic representations, it can naturally be extended to graph-based representations as well.

As discussed previously, the learning problem reduces to identifying two types of structural changes: deassociation, where a matrix entry changes from $1 \rightarrow 0$, and reassociation, where a matrix entry changes from $0 \rightarrow 1$. The corresponding learning goal is therefore to identify the minimal environment around the changing cell that is both sufficient and exclusive in triggering the change observed in the positive evidence.

$$\begin{array}{ccc} & ? & ? & ? \\ 0 \rightarrow 1 / & ? & _ & ? \\ & ? & ? & ? \end{array} \quad \text{or} \quad \begin{array}{ccc} & ? & ? & ? \\ 1 \rightarrow 0 / & ? & _ & ? \\ & ? & ? & ? \end{array}$$

To learn ASGs, we adapt the bottom-up factor inference framework of [Chandlee et al. \(2019\)](#) and the logical learning framework of [Yolyan \(2025\)](#) to graph-based autosegmental representations **[NEED REF]**. At the beginning of learning, the learner first identifies changing association sites by comparing the input and output association matrices. If an association value changes from $1 \rightarrow 0$, the change is classified as deassociation; if it changes from $0 \rightarrow 1$, it is classified as reassociation. The learner then initializes the search with the corresponding most general logical hypothesis: $\neg\alpha(x, y)$ for reassociation and $\exists x \exists y \alpha(x, y)$ for deassociation ¹.

Data: $D = \{(I_1, O_1), \dots, (I_n, O_n)\}, k, V^+ = \{I_n \mid I_n \neq O_n\}, V^- = \{I_n \mid I_n = O_n\}$

```

1  $P \leftarrow \emptyset$ 
2  $Q \leftarrow \{\alpha(x, y)\}$  or  $Q \leftarrow \{\neg\alpha(x, y)\}$ 
3 while  $Q \neq \emptyset$  do
4    $c \leftarrow Q.\text{Dequeue}()$ 
5   if  $\forall g \in V^+, c \sqsubseteq g$  then
6     if  $\forall g \in V^-, c \not\sqsubseteq g$  then
7       return  $c$ 
8     else
9       foreach  $s \in \text{NEXTSUPFACT}(c) \wedge |s| < k \wedge \forall p \in P, s \not\sqsubseteq p$  do
10         $Q.\text{Enqueue}(s)$ 
11   else
12      $P \leftarrow P \cup \{c\}$ 
13 return  $\perp$ 

```

Algorithm 1: Learning Association Changes

1. We treat deassociation and reassociation as two separate learning problems and combine the resulting hypotheses through disjunction.

The input data consists of a finite set of input-output ASG pairs, which can be partitioned into positive and negative evidence, denoted V^+ and V^- respectively. It also requires a bound k that limits the graph size.

The algorithm conducts a breadth-first search over a partially ordered hypothesis space. At each step, a candidate hypothesis c is tested against the positive evidence: if c is not contained in all positive examples, then the current hypothesis is incompatible with the observed transformation and thus is pruned. Otherwise, the learner checks c against all negative examples: if none of data contains c , then we return the current structure, otherwise, it is considered too general. The learner therefore generates more specific superfactors using NEXTSUPFACT. Because each application of NEXTSUPFACT strictly increases the size of the hypothesis and the search is bounded by k , the hypothesis space is finite and the algorithm terminates. Moreover, since hypotheses are explored breadth-first, the first valid hypothesis returned by the learner is guaranteed to be the minimal structural environment consistent with all positive and negative evidence. One crucial component of the learning procedure is NEXTSUPFACT. There are many possible ways to generate superfactors while still covering the same hypothesis space. Here, we propose one possible method that explicitly connects graph-theoretic representations with their logical counterparts.

From a graph-theoretic perspective, the default first step is to expand the domain by introducing neighboring nodes. Such neighboring nodes may be added on either tier in any successor relation to existing nodes. Table 2 presents a non-exhaustive set of examples illustrating the generation of NEXTSUPFACT hypotheses starting from an initial deassociation hypothesis. By adding neighbor, one can expect a possible candidate with an additional node z following x . Other possible configurations include x following z , z following x , z following y , or y following z . Once additional nodes are introduced and valid association docking sites become available, new association lines may be added between nodes. Finally, the learner may further specialize the hypothesis by specifying node labels or feature values ².

As can be seen, this provides a clear correspondence between graph-theoretic and logical representations. New quantifiers are introduced through successor relations, corresponding to the addition of neighboring nodes in the graph structure. Within the same quantifier scope, the learner may further specialize the hypothesis through conjunction, including adding association relations (corresponding to changing matrix cells) or specifying node labels and feature values.

3. Case Study

3.1. Contour Formation

4. Conclusion

This paper has argued that vowel-deletion-induced tonal processes can be uniformly captured as logical transductions over autosegmental representations (ARs). The observed typological variation follows from modifying a single relation, α , through the interaction of deassoc and reassoc.

2. For a more detailed description of NEXTSUPFACT operations over ASGs, see [NEED REF].

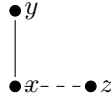
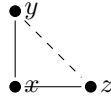
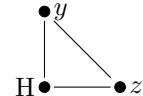
Operation	Matrix	Graph	Logic
Initial hypothesis	$\frac{}{y} \mid \frac{x}{1}$		$\alpha(x, y)$
Add neighbor	$\frac{}{y} \mid \frac{x \quad z}{1}$		$\alpha(x, y) \wedge \exists z \triangleleft (x, z)$
Add association	$\frac{}{y} \mid \frac{x \quad z}{1 \quad 1}$		$\alpha(x, y) \wedge \exists z (\triangleleft (x, z) \wedge \alpha(y, z))$
Add label	$\frac{}{y} \mid \frac{H \quad z}{1 \quad 1}$		$\alpha(x, y) \wedge \exists z (\triangleleft (x, z) \wedge \alpha(y, z) \wedge H(x))$

Table 2: Examples of operations used in generating NEXTSUPFACT hypotheses across matrix, graph, and logical representations.

A limitation of the current analysis is that the transduction relies on quantification to capture across-tier dependencies. An important direction for future work is to determine whether these processes can be simplified in a quantifier-free (QF) process. This work also extends naturally to learning over ARs. Recent work by Yolyan (2025) demonstrates that logical transductions are learnable by using a bottom-up factorial inference algorithms (Chandlee et al., 2019). This opens up more discussions about how linguistically meaningful representations can facilitate grammar inference.

References

- Steven Bird and T Mark Ellison. One-level phonology: Autosegmental representations and rules as finite automata. *Computational Linguistics*, 20(1):55–90, 1994.
- Jane Chandlee, Rémi Eyraud, Jeffrey Heinz, Adam Jardine, and Jonathan Rawski. Learning with partially ordered representations. In *Proceedings of MOL16*, pages 91–101, 2019.
- John Anton Goldsmith. *Autosegmental phonology*. PhD thesis, Massachusetts Institute of Technology, 1976.
- Bruce Philip Hayes. *A metrical theory of stress rules*. PhD thesis, Massachusetts Institute of Technology, 1980.
- Adam Jardine. The local nature of tone-association patterns. *Phonology*, 34(2):363–384, 2017.
- Jonathan Rawski. *Structure and learning in natural language*. PhD thesis, State University of New York at Stony Brook, 2021.

- Elizabeth Caroline Sagey. *The representation of features and relations in non-linear phonology*. PhD thesis, Massachusetts Institute of Technology, 1986.
- Kristina Strother-Garcia. *Using model theory in phonology: a novel characterization of syllable structure and syllabification*. University of Delaware, 2019.
- Bruce Wiebe. Modelling autosegmental phonology with multi-tape finite state transducers. 1992.
- Anssi Yli-Jyrä. On finite-state tonology with autosegmental representations. In *Proceedings of the 11th international conference on finite state methods and natural language processing*, pages 90–98, 2013.
- Anssi Yli-Jyrä. Three equivalent codes for autosegmental representations. In *Proceedings of the 12th International Conference on Finite-State Methods and Natural Language Processing 2015 (FSMNLP 2015 Düsseldorf)*, 2015.
- Tatevik Yolyan. A framework for learning phonological maps as logical transductions. In *Proceedings of MOL18*, pages 44–58, 2025.